



# Manual SIGAA

Instalación, desarrollo  
y colaboración

Docker · Git · GitHub · Código · Seguridad

**Proyecto Capstone 2026**

Versión 1.0 · 24 de agosto de 2026

# Antes de comenzar

Este manual permite instalar SIGAA, trabajar en el código y publicar cambios sin acceder al Mac servidor ni a su base de datos.

## Regla principal

Para desarrollar se usa PostgreSQL local dentro de Docker. El servidor compartido se utiliza solamente mediante la URL HTTPS autorizada.

## Contenido

| Sección | Tema                     | Resultado                                   |
|---------|--------------------------|---------------------------------------------|
| 01      | Requisitos e instalación | Preparar Windows, macOS o Linux             |
| 02      | Clonar el repositorio    | Descargar la rama correcta                  |
| 03      | Ejecutar con Docker      | Iniciar web, API y base local               |
| 04      | Modificar el código      | Ubicar frontend, API, datos y documentación |
| 05      | Subir cambios            | Ramas, commits, push y Pull Request         |
| 06      | Seguridad y soporte      | Evitar secretos y resolver errores          |

|   |                                                  |
|---|--------------------------------------------------|
| ✓ | Instalar Git, Docker y Visual Studio Code.       |
| ✓ | Clonar el repositorio y abrir fase-1-evidencias. |
| ✓ | Levantar Docker Compose.                         |
| ✓ | Crear una rama propia antes de editar.           |
| ✓ | Probar, hacer commit, push y Pull Request.       |

# Cómo se trabaja en SIGAA

Cada integrante posee una copia reproducible. Docker crea los mismos servicios sin instalar Node.js ni PostgreSQL en el computador.



| Etapa        | Qué ocurre                        | Herramienta    |
|--------------|-----------------------------------|----------------|
| 1. Descargar | Git copia archivos e historial.   | Git + GitHub   |
| 2. Editar    | Se trabaja en una rama propia.    | VS Code        |
| 3. Ejecutar  | Docker inicia los tres servicios. | Docker Compose |
| 4. Probar    | Se revisa en localhost.           | Navegador      |
| 5. Compartir | Se abre un Pull Request.          | GitHub         |

**No trabajan sobre producción**

Los cambios locales no afectan el Mac servidor. El despliegue ocurre después de la revisión.

# 01 · Requisitos e instalación

Solo son obligatorios Git, Docker y un editor. Node.js, React, Nginx y PostgreSQL se ejecutan en contenedores.

| Herramienta             | Uso                   | Comprobación                   |
|-------------------------|-----------------------|--------------------------------|
| Git                     | Descargar y versionar | git --version                  |
| Docker Desktop / Engine | Ejecutar SIGAA        | docker --version               |
| Docker Compose          | Coordinar servicios   | docker compose version         |
| Visual Studio Code      | Editar el proyecto    | code --version                 |
| Navegador moderno       | Probar SIGAA          | Chrome, Edge, Firefox o Safari |

## Windows 10/11

- Instalar Git desde <https://git-scm.com/download/win>.
- Instalar Docker Desktop desde la documentación oficial.
- Habilitar WSL 2 si Docker lo solicita y reiniciar.
- Instalar VS Code desde <https://code.visualstudio.com/Download>.

## macOS

- Instalar Git o Xcode Command Line Tools.
- Instalar Docker Desktop para Apple Silicon o Intel.
- Instalar VS Code.

## Linux

- Instalar Docker Engine y el complemento Compose desde el repositorio oficial.
- Cerrar sesión después de modificar permisos del grupo docker.

# Verificar la instalación

Abre PowerShell, Terminal o Windows Terminal y ejecuta cada comando por separado.

```
git --version
docker --version
docker compose version
code --version
```

|   |                                               |
|---|-----------------------------------------------|
| ✓ | Git muestra una versión.                      |
| ✓ | Docker Desktop está iniciado.                 |
| ✓ | Compose responde con versión 2.x o posterior. |
| ✓ | VS Code abre o está disponible desde el menú. |

## Si Docker no responde

Inicia Docker Desktop y espera a que el motor esté listo. No continúes si aparece Cannot connect to the Docker daemon.

## Recursos recomendados

| Recurso            | Mínimo           | Recomendado      |
|--------------------|------------------|------------------|
| RAM libre          | 4 GB             | 6 GB o más       |
| Espacio disponible | 5 GB             | 10 GB o más      |
| Procesador         | 64 bits          | 4 núcleos        |
| Internet           | Primera descarga | Conexión estable |

## 02 · Clonar el repositorio

Joel debe invitar como colaborador a quien necesite subir ramas. Descargar un repositorio público no requiere permiso.

**1**

### Confirmar GitHub

Inicia sesión, envía tu nombre de usuario al Product Owner y acepta la invitación de colaborador.

**2**

### Elegir una carpeta

Usa Documentos/Proyectos. Evita carpetas sincronizadas que puedan pausar archivos de Docker o Git.

**3**

### Clonar y cambiar de rama

Copia exactamente la URL. El repositorio se llama Cptsone.

```
git clone https://github.com/joellopez775/Cptsone.git
cd Cptsone
git switch fase-1-evidencias
cd SIGAA
```

## Comprobar

```
git branch --show-current
git remote -v
git status
```

**Resultado esperado**

La rama es fase-1-evidencias, origin apunta a joellopez775/Cptsone y no hay cambios locales.

## 03 · Configurar y ejecutar Docker

Ejecuta estos comandos dentro de Captstone/SIGAA.

### Crear la configuración local

El archivo .env es privado y está excluido de Git.

#### macOS, Linux o Git Bash

```
cp .env.example .env
```

#### Windows PowerShell

```
Copy-Item .env.example .env
```

### Levantar SIGAA

```
docker compose up --build --detach
docker compose ps
```

#### Primera ejecución

Docker descargará imágenes y construirá el proyecto. Puede tardar varios minutos.

| Servicio    | Función           | Estado  | Acceso          |
|-------------|-------------------|---------|-----------------|
| sigaa-web-1 | React + Nginx     | healthy | localhost:8088  |
| sigaa-api-1 | Node.js + Express | healthy | localhost:3000  |
| sigaa-db-1  | PostgreSQL 16     | healthy | Solo red Docker |

# Abrir y comprobar SIGAA

Cuando los tres servicios indiquen healthy, abre <http://localhost:8088>.

| Perfil     | Correo                | Contraseña      |
|------------|-----------------------|-----------------|
| Docente    | docente@sigaa.demo    | Docente2026!    |
| Estudiante | estudiante@sigaa.demo | Estudiante2026! |

## Pruebas rápidas

```
docker compose ps
curl http://localhost:3000/health
curl http://localhost:8088/api/health
```

|   |                                                   |
|---|---------------------------------------------------|
| ✓ | La pantalla de acceso carga.                      |
| ✓ | Docente abre cursos, clases, notas y anotaciones. |
| ✓ | Estudiante abre notas, asistencia y calendario.   |
| ✓ | Solo se utilizan datos sintéticos.                |

## Detener sin borrar datos

```
docker compose down
```

### Cuidado

`docker compose down --volumes` elimina la base local. Úsalo solo si aceptaste perder esos datos.



# 04 · Acceder y modificar el código

Abre la carpeta SIGAA completa en Visual Studio Code.

```
code .
```

| Ruta                           | Responsabilidad   | Cambios típicos            |
|--------------------------------|-------------------|----------------------------|
| apps/web/src/App.jsx           | Pantallas React   | Vistas y navegación        |
| apps/web/src/styles.css        | Diseño responsivo | Colores y móvil            |
| apps/api/src/app.js            | API HTTP          | Endpoints y validación     |
| apps/api/src/prototype-data.js | Datos sintéticos  | Ejemplos demo              |
| apps/api/test/                 | Pruebas           | Seguridad y respuestas     |
| infra/postgres/init/           | Migraciones       | Tablas e índices           |
| docs/                          | Trazabilidad      | Diseño, backlog, bitácoras |
| compose.yaml                   | Entorno local     | Servicios y redes          |

No editar

No modificar node\_modules, dist, .env, secretos, claves SSH, respaldos ni datos reales.

# Crear una rama antes de editar

Una rama propia permite revisar el trabajo sin alterar la línea base.

```
git switch fase-1-evidencias
git pull --ff-only
git switch -c feature/tu-nombre-descripcion-corta
```

## Ejemplos

```
feature/matias-mejorar-asistencia
feature/agustin-validar-formulario
docs/joel-manual-usuario
fix/maria-menu-movil
```

|   |                                                        |
|---|--------------------------------------------------------|
| ✓ | feature/: funcionalidad nueva.                         |
| ✓ | fix/: corrección.                                      |
| ✓ | docs/: documentación.                                  |
| ✓ | Usar minúsculas y guiones.                             |
| ✓ | No trabajar directamente en main ni fase-1-evidencias. |

**Antes de cada tarea** Actualiza fase-1-evidencias y después crea la rama. Esto reduce conflictos.

# Ciclo seguro de modificación

Haz cambios pequeños, comprensibles y verificables.

**1**

## Editar

Modifica solo los archivos necesarios y conserva la experiencia responsiva.

**2**

## Reconstruir

Si cambias React, API o Nginx, reconstruye para evitar una imagen antigua.

```
docker compose up --build --detach
```

**3**

## Probar

Valida Compose, ejecuta las pruebas API y construye la web.

```
docker compose config --quiet
docker compose run --rm api npm test
docker compose build web
```

**4**

## Revisar

Comprueba exactamente qué cambió.

```
git status --short
git diff
```

**Documentar  
siempre**

Todo avance material debe actualizar la bitácora, backlog o documento técnico correspondiente.

# Preparar y confirmar cambios

Agrega archivos explícitos para no incluir secretos o trabajo ajeno.

## Agregar archivos concretos

```
git add -- apps/web/src/App.jsx apps/web/src/styles.css
git status --short
```

## Crear el commit

```
git commit -m "feat: mejorar vista de asistencia"
```

| Prefijo  | Uso            | Ejemplo                             |
|----------|----------------|-------------------------------------|
| feat     | Funcionalidad  | feat: agregar filtro por curso      |
| fix      | Corrección     | fix: corregir menú móvil            |
| docs     | Documentación  | docs: actualizar manual Docker      |
| test     | Pruebas        | test: cubrir acceso cruzado         |
| refactor | Mejora interna | refactor: separar portal estudiante |

### Nunca confirmar

.env, .env.server, .secrets/, claves SSH, tokens, contraseñas, SQL, node\_modules, dist o datos reales.

## 05 · Subir cambios al repositorio

El push publica tu rama; el Pull Request solicita revisión antes de mezclarla.

### Primera subida

```
git push -u origin feature/tu-nombre-descripcion-corta
```

### Crear el Pull Request

|   |                                                                                                           |
|---|-----------------------------------------------------------------------------------------------------------|
| ✓ | Abrir <a href="https://github.com/joellopez775/Captstone">https://github.com/joellopez775/Captstone</a> . |
| ✓ | Seleccionar Compare & pull request.                                                                       |
| ✓ | Base: fase-1-evidencias.                                                                                  |
| ✓ | Compare: tu rama.                                                                                         |
| ✓ | Describir cambio, pruebas, evidencia y pendientes.                                                        |
| ✓ | Solicitar revisión y esperar aprobación.                                                                  |

### Plantilla

```
## Qué cambia
- ...

## Cómo se probó
- docker compose config --quiet
- recorrido manual de ...

## Evidencia
- captura / documento / resultado

## Riesgos o pendientes
- ...
```

# Actualizar una rama y resolver observaciones

Las correcciones solicitadas se suben a la misma rama y actualizan automáticamente el Pull Request.

```
# Editar y probar nuevamente
git add -- archivo-modificado
git commit -m "fix: aplicar observaciones de revisión"
git push
```

## Incorporar cambios recientes

```
git fetch origin
git merge origin/fase-1-evidencias
```

## Si aparece un conflicto

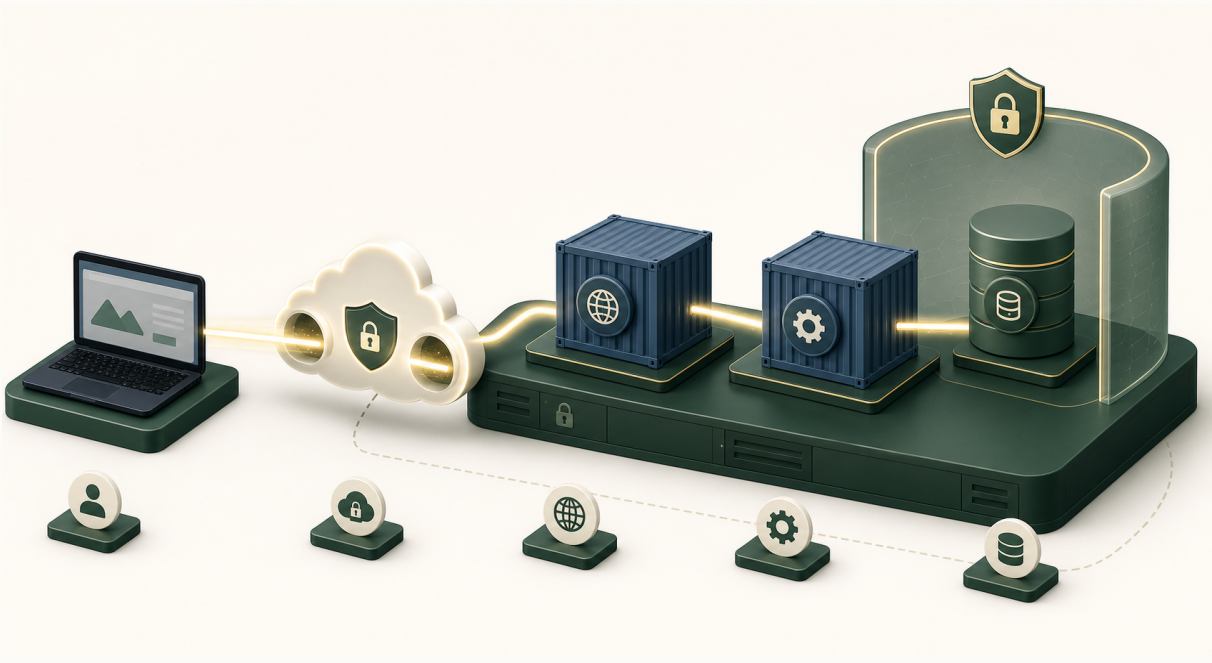
|   |                                                |
|---|------------------------------------------------|
| ✓ | No usar git reset --hard.                      |
| ✓ | Revisar ambos bloques en cada archivo.         |
| ✓ | Integrar las intenciones correctas.            |
| ✓ | Ejecutar de nuevo las pruebas.                 |
| ✓ | Marcar con git add y crear un commit.          |
| ✓ | Solicitar ayuda si el conflicto no está claro. |

### Prohibido

No usar git push --force en ramas compartidas ni fusionar tu propio Pull Request sin autorización.

# 06 · Acceso seguro y aislamiento

Los compañeros acceden al portal HTTPS y a GitHub, no al Mac ni a PostgreSQL.



| Recurso      | Acceso del compañero | Protección                 |
|--------------|----------------------|----------------------------|
| Portal SIGAA | URL HTTPS            | Cloudflare Tunnel + Access |
| Código       | GitHub y ramas       | Permisos + Pull Request    |
| Mac servidor | Ninguno              | Sin SSH o Tailscale        |
| PostgreSQL   | Ninguno              | Red Docker interna         |
| Base local   | Completo en su PC    | Volumen independiente      |

Colaborar en código no requiere entregar acceso al computador de producción.

### Datos personales reales

SIGAA todavía usa cuentas demo. Debe permanecer detrás de Cloudflare Access y contener solo datos sintéticos.

Solo el propietario despliega.



# Solución de problemas

Envía el error completo al solicitar ayuda, ocultando cualquier secreto.

| Problema             | Causa común                 | Solución segura                         |
|----------------------|-----------------------------|-----------------------------------------|
| docker no responde   | Docker detenido             | Iniciar Docker y repetir docker version |
| Puerto 8088 ocupado  | Otro proceso                | Cambiar WEB_PORT en .env                |
| Cambios no aparecen  | Imagen antigua              | docker compose up --build --detach      |
| Contenedor unhealthy | Error interno               | Revisar docker compose logs             |
| Push rechazado       | Sin permiso o rama atrasada | Aceptar invitación; fetch y merge       |
| .env en Git          | Se agregó por error         | No hacer push; retirar y rotar secretos |
| Conflicto Git        | Mismas líneas cambiadas     | Resolver con revisión; no reset --hard  |

## Comandos de diagnóstico

```
docker compose ps
docker compose logs --tail=100 web
docker compose logs --tail=100 api
docker compose logs --tail=100 db
git status --short
git log -5 --oneline
```

# Checklist antes del Pull Request

El cambio debe ser comprensible, reproducible y seguro.

|   |                                                |
|---|------------------------------------------------|
| ✓ | Estoy en mi rama.                              |
| ✓ | El cambio responde a una tarea concreta.       |
| ✓ | Docker Compose valida.                         |
| ✓ | Los servicios están healthy.                   |
| ✓ | Probé el flujo afectado.                       |
| ✓ | Las pruebas pasan.                             |
| ✓ | No incluí secretos ni datos reales.            |
| ✓ | Actualicé documentación y bitácora.            |
| ✓ | El commit explica el resultado.                |
| ✓ | La base del Pull Request es fase-1-evidencias. |
| ✓ | Describí pruebas, evidencia y pendientes.      |

## Definition of Done

Compilar no basta: el cambio debe estar probado, documentado, revisable y sin secretos.

# Referencia rápida

Guarda esta página para el trabajo cotidiano.

| Objetivo        | Comando                                                                 |
|-----------------|-------------------------------------------------------------------------|
| Actualizar base | <code>git switch fase-1-evidencias &amp;&amp; git pull --ff-only</code> |
| Crear rama      | <code>git switch -c feature/nombre-cambio</code>                        |
| Levantar        | <code>docker compose up --build --detach</code>                         |
| Estado Docker   | <code>docker compose ps</code>                                          |
| Ver cambios     | <code>git status --short &amp;&amp; git diff</code>                     |
| Agregar         | <code>git add -- archivo1 archivo2</code>                               |
| Commit          | <code>git commit -m "feat: descripción"</code>                          |
| Subir           | <code>git push -u origin nombre-rama</code>                             |
| Detener         | <code>docker compose down</code>                                        |

## Enlaces oficiales

- Repositorio: <https://github.com/joellopez775/Captstone>
- Git: <https://git-scm.com/downloads>
- Docker: <https://docs.docker.com/desktop/>
- Docker Engine: <https://docs.docker.com/engine/install/>
- VS Code: <https://code.visualstudio.com/Download>
- GitHub Pull Requests: <https://docs.github.com/pull-requests>
- Cloudflare Tunnel: <https://developers.cloudflare.com/tunnel/>

### Para pedir ayuda

Incluye sistema operativo, comando, salida del error y `git status --short`. Oculta secretos.